

HARPIa - Guia de Estudo do Código YOLO

Este guia é complementar ao relatório técnico. O foco é explicar **como a implementação funciona e onde cada decisão aparece no código**.

Mapa do repositório

Arquivo	Responsabilidade
<code>inference.py</code>	Inferência em imagem estática, bbox, centro e erro visual
<code>scripts/detect_people.py</code>	Pipeline principal de vídeo
<code>scripts/run_video_poc.sh</code>	Orquestração test/full/report e logs
<code>scripts/watch_video_log.sh</code>	Acompanhamento ao vivo com <code>tail -f</code>
<code>scripts/transcode_web_video.py</code>	Conversão do MP4 anotado para H.264
<code>scripts/video_report.py</code>	Geração do painel HTML
<code>scripts/download_video.sh</code>	Download reproduzível da entrada

Fluxo mental do detector

```
CLI
-> valida argumentos
-> carrega OpenCV + YOLO
-> procura classe person
-> abre vídeo
-> cria VideoWriter
-> loop frame a frame
    -> model.predict
    -> extrai box/conf
    -> desenha bbox
    -> grava frame anotado
    -> atualiza métricas
    -> atualiza melhores exemplos
-> libera recursos
-> calcula métricas finais
-> salva JSON
```

CLI e parâmetros

`build_parser()` transforma o detector em uma ferramenta reutilizável.

```
DEFAULT_MODEL = "yolo11n.pt"
DEFAULT_IMGSZ = 640
DEFAULT_CONF = 0.25
DEFAULT_DEVICE = "cpu"
```

```
parser.add_argument("--input", type=Path, required=True)
parser.add_argument("--output", type=Path, required=True)
parser.add_argument("--model", default=DEFAULT_MODEL)
parser.add_argument("--imgsz", type=int, default=DEFAULT_IMGSZ)
parser.add_argument("--conf", type=float, default=DEFAULT_CONF)
parser.add_argument("--device", default=DEFAULT_DEVICE)
parser.add_argument("--max-frames", type=int, default=None)
```

Como explicar:

- `imgsz`: resolução interna usada pelo modelo;
- `conf`: limiar mínimo de confiança;
- `device`: CPU no ambiente validado;
- `max-frames`: permite smoke tests curtos;
- `metrics`: define o JSON de saída;
- `examples`: define quantos frames representativos serão preservados.

Validação antecipada

`validate_args()` falha cedo quando a entrada ou parâmetros são inválidos.

```
if not args.input.is_file():
    raise ValueError(...)
if not 0.0 <= args.conf <= 1.0:
    raise ValueError(...)
if args.imgsz <= 0:
    raise ValueError(...)
```

Isso evita carregar modelo e dependências antes de descobrir um erro simples de uso.

Dependências e NNPACK

`load_dependencies()` importa OpenCV, PyTorch e Ultralytics somente quando a inferência realmente começa.

```
import cv2
import torch
from ultralytics import YOLO

if hasattr(torch.backends, "nnpack"):
    torch.backends.nnpack.set_flags(False)
```

O tratamento de NNPACK existe porque a CPU antiga do HARPIa não suporta esse backend de otimização. Isso não desativa a inferência em CPU.

Descoberta programática da classe person

O código evita usar `person = 0` como número mágico.

```
def find_person_class_id(names):
    if isinstance(names, dict):
        items = names.items()
    else:
        items = enumerate(names)

    matches = [
        int(class_id)
        for class_id, class_name in items
        if str(class_name).strip().lower() == "person"
    ]
```

Depois:

```
person_class_id = find_person_class_id(model.names)
```

E o filtro entra na própria inferência:

```
results = model.predict(
    source=frame,
    imsz=args.imsz,
    conf=args.conf,
    device=args.device,
    classes=[person_class_id],
    verbose=False,
)
```

Por que isso é melhor: o código expressa a intenção pelo nome da classe e não depende silenciosamente da indexação do COCO.

Abertura do vídeo

```
cap = cv2.VideoCapture(str(args.input))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
original_fps = float(cap.get(cv2.CAP_PROP_FPS))
source_frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

Esses valores servem para validar a mídia, documentar a execução e configurar a saída.

Por que a saída mantém 29,97 FPS

O writer usa `original_fps`:

```
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter(
    str(args.output),
    fourcc,
    original_fps,
    (width, height),
)
```

O computador processou 2,531 FPS na execução completa, mas esse número mede velocidade de cálculo. O arquivo é gravado com o FPS original e reproduz na duração normal.

Loop principal

```
while True:
    if args.max_frames is not None and frames_processed >= args.max_frames:
        break

    ok, frame = cap.read()
    if not ok:
        break

    results = model.predict(...)
    result = results[0]
```

O fluxo é deliberadamente explícito: OpenCV entrega um frame, YOLO processa, o código extrai detecções e gera a saída.

Tempo de inferência x tempo total

O Ultralytics disponibiliza tempos internos em `result.speed`.

```
speed = getattr(result, "speed", {}) or {}
inference_ms = speed.get("inference")
```

- `mean_yolo_inference_ms`: tempo médio reportado pelo YOLO;

- `mean_ms_per_frame`: tempo do pipeline completo, incluindo leitura, desenho e escrita.

Na execução completa:

- YOLO: 354,935 ms/frame;
- pipeline completo: 395,077 ms/frame.

Extração das boxes

```
frame_detections = []

if result.frames is not None:
    for box in result.frames:
        class_id = int(box.cls[0])
        if class_id != person_class_id:
            continue

        confidence = float(box.conf[0])
        bbox = [float(value) for value in box.xyxy[0].tolist()]
        frame_detections.append((bbox, confidence))
```

`xyxy` significa `x1, y1, x2, y2`.

A checagem da classe dentro do loop é uma defesa extra, mesmo que `classes=[person_class_id]` já tenha sido enviado ao modelo.

Onde as bounding boxes são desenhadas

`draw_detection()` recebe `frame`, `bbox` e confiança.

```
x1, y1, x2, y2 = [int(round(value)) for value in bbox]
label = f"person {confidence:.2f}"
```

```
cv2.rectangle(
    frame,
    (x1, y1),
    (x2, y2),
    (0, 255, 0),
    2,
)
```

Depois o label é escrito com `cv2.putText()`.

No loop:

```
annotated = frame.copy()

for bbox, confidence in frame_detections:
    draw_detection(cv2, annotated, bbox, confidence)
    confidences.append(confidence)

writer.write(annotated)
```

Resposta pronta para apresentação: `draw_detection()` **altera** `annotated`, e `writer.write(annotated)` **grava exatamente esse frame no vídeo**.

Contagem de pessoas no frame

```
people_this_frame = len(frame_detections)
total_person_detections += people_this_frame
```

O texto de status também é desenhado no frame:

```
cv2.putText(
    annotated,
    f"frame={frame_index} | persons={people_this_frame}",
    ...
)
```

`total_person_detections` não representa indivíduos únicos. A mesma pessoa pode aparecer em centenas de frames.

Seleção dos frames de evidência

O script mantém somente os melhores N frames em um heap.

```
confidence_sum = sum(
    confidence for _, confidence in frame_detections
)
score = (float(people_this_frame), float(confidence_sum))

if len(examples_heap) < args.examples:
    heapq.heappush(examples_heap, heap_item)
elif score > examples_heap[0][0]:
    heapq.heapreplace(examples_heap, heap_item)
```

Critério:

- 1 maior número de pessoas;
- 2 em empate, maior soma das confidências.

Por que heap: a memória depende apenas de N exemplos, não de 1137 frames.

Cálculo das métricas

```
processing_seconds = time.perf_counter() - processing_start
processing_fps = frames_processed / processing_seconds
mean_ms_per_frame = processing_seconds * 1000.0 / frames_processed
output_duration_seconds = frames_processed / original_fps
```

Estatísticas de confiança:

```
mean_confidence = mean(confidences) if confidences else None
min_confidence = min(confidences) if confidences else None
max_confidence = max(confidences) if confidences else None
```

O JSON agrupa:

- modelo e classe;
- vídeo;
- parâmetros;
- timing;
- detecções;
- exemplos.

Liberação de recursos

```
try:
    while True:
        ...
finally:
    cap.release()
    writer.release()
```

Mesmo se algo falhar dentro do loop, OpenCV libera os recursos.

Exit codes

`main()` transforma falhas esperadas em códigos de retorno previsíveis:

```
except (ValueError, RuntimeError) as exc:
    print(f"ERRO: {exc}", file=sys.stderr)
    return 1
except KeyboardInterrupt:
    return 130
```

Isso permite que Bash, CI ou outro processo detectem falhas corretamente.

inference.py - imagem estática

O fluxo estático usa a mesma ideia, mas processa uma única imagem.

```
model = YOLO(args.model)
results = model.predict(
    source=str(args.image),
    imgsz=args.imgsz,
    conf=args.conf,
    device=args.device,
    verbose=False,
)
```

Além da bbox, ele calcula centro e erro em relação ao centro da imagem:

```
cx = (x1 + x2) / 2.0
cy = (y1 + y2) / 2.0
ex = cx - (image_width / 2.0)
ey = cy - (image_height / 2.0)
```

`ex` e `ey` podem ser úteis em trabalhos futuros de alinhamento visual, mas não estão conectados ao PX4 nesta POC.

run_video_poc.sh

O runner possui três modos:

Modo	Comportamento
test	60 frames, log, vídeo curto, H.264 e painel
full	vídeo completo, métricas, evidências, H.264 e painel
report	não roda YOLO; reutiliza resultados para H.264 + painel

Trecho conceitual:

```
case "$MODE" in
    full) ... ;;
```

```
test) ... --max-frames 60 ... ;;
report)
    RUN_DETECTOR=0
    ;;
esac
```

tee e PIPESTATUS

Em Bash:

```
python ... 2>&1 | tee -a "$LOG"
STATUS=${PIPESTATUS[0]}
```

`PIPESTATUS[0]` preserva o código de retorno do Python, em vez de confundir o sucesso do `tee` com o sucesso do detector.

watch_video_log.sh

O watcher procura o log mais recente e executa:

```
tail -n 40 -f "$LATEST"
```

Operação recomendada:

- terminal 1: `run_video_poc.sh test` ou `full`;
- terminal 2: `watch_video_log.sh`;
- navegador: painel HTML.

transcode_web_video.py

O detector gera MP4 com `mp4v`, que pode não tocar no navegador. A solução converte o vídeo já anotado:

```
command = [
    ffmpeg,
    "-i", str(input_path),
    "-map", "0:v:0",
    "-c:v", "libx264",
    "-preset", "fast",
    "-crf", str(crf),
    "-pix_fmt", "yuv420p",
    "-movflags", "+faststart",
    "-an",
    str(output_path),
]
```

Pontos para explicar:

- `libx264`: H.264 compatível com navegador;
- `yuv420p`: ampla compatibilidade de pixel format;
- `+faststart`: facilita início progressivo;
- `-an`: remove áudio;
- `CRF 20`: equilíbrio entre qualidade e tamanho.

As bounding boxes são preservadas porque já foram desenhadas nos pixels antes da conversão.

video_report.py

O painel é consumidor de artefatos. Ele não executa YOLO.

```
metrics = load_json(args.metrics)
source = load_json(args.source) if args.source.is_file() else {}
comparison = read_comparison(args.comparison)
```

Depois `render_report()` monta o HTML com:

- cards de métricas;
- vídeo anotado;
- evidências;
- benchmark;
- trecho do log.

O vídeo é referenciado localmente no HTML.

download_video.sh

O script registra a fonte da POC e procura `yt-dlp` nesta ordem:

- 1 variável `YT_DLP`;
- 2 `yt-dlp` disponível no `PATH`;
- 3 `/root/yolo_venv/bin/yt-dlp`.

O vídeo é baixado para `input/people_cc.mp4` e não é versionado.

Como explicar tudo em 90 segundos

- 1 A CLI define entrada e parâmetros.
- 2 YOLO11n é carregado em CPU.
- 3 O código procura `person` em `model.names`.
- 4 OpenCV entrega um frame por vez.
- 5 YOLO retorna boxes e confiança.
- 6 `draw_detection()` desenha cada box.
- 7 `writer.write(annotated)` cria o MP4 anotado.
- 8 O script acumula métricas e salva JSON.
- 9 Um heap mantém os melhores frames de evidência.
- 10 O runner adiciona logs, H.264 e painel sem misturar percepção com controle.

Perguntas técnicas prováveis

Por que não usar `person = 0`?

Para reduzir acoplamento à indexação do dataset e expressar a intenção pelo nome da classe.

Onde a confiança é aplicada?

Em `model.predict(conf=args.conf)`. Depois `box.conf` é preservado para label e métricas.

Onde os boxes entram no vídeo?

`draw_detection()` altera `annotated` e `writer.write(annotated)` grava o frame.

Por que copiar o frame?

Para manter o frame original separado do frame de apresentação.

Por que usar heap?

Para manter somente N melhores exemplos em memória.

Por que `processing_fps` e `mean_yolo_inference_ms` contam histórias diferentes?

O primeiro mede o pipeline completo; o segundo mede somente a inferência reportada pelo Ultralytics.

MP4 e H.264 são a mesma coisa?

Não. MP4 é container; H.264 e MP4V são codecs.

O modo `report` roda YOLO?

Não. Ele reutiliza vídeo e métricas existentes.

Por que `result.plot()` não é usado no vídeo?

No vídeo, o projeto desenha manualmente para controlar label e tornar a lógica explícita. No fluxo estático, `result.plot()` é usado.

Roteiro para abrir o código na apresentação

Ordem	Trecho	Mensagem
1	<code>find_person_class_id</code>	classe sem número mágico
2	<code>model.predict</code>	parâmetros e filtro person
3	<code>draw_detection</code>	onde a bbox é desenhada
4	<code>writer.write</code>	ligação entre frame e vídeo
5	dicionário <code>metrics</code>	execução vira dados auditáveis
6	<code>run_video_poc.sh</code>	operação, logs e modos
7	<code>transcode_web_video.py</code>	compatibilidade do navegador

Ordem	Trecho	Mensagem
8	video_report.py	painel consome artefatos

Checklist de estudo

- explicar diferença entre `inference.py` e `detect_people.py`;
- localizar `person_class_id`;
- localizar `bbox` e `writer.write`;
- explicar `original_fps` x `processing_fps`;
- explicar por que 7044 não são pessoas únicas;
- explicar o heap de evidências;
- explicar timing do pipeline x timing do YOLO;
- dominar modos `test`, `full` e `report`;
- explicar por que H.264 preserva as boxes;
- deixar claro que não existe controle PX4 neste repositório.

Use este guia para responder **como foi implementado**. Use o Relatório Técnico para responder **o que foi feito, por que e quais resultados foram obtidos**.